同步读写的客户端和服务器示例

Posted on 2023-04-02 12:06:25 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Nr3U2vU2UxYM0odYRGAk3wWO1S)

简介

前面我们介绍了boost::asio同步读写的api函数，现在将前面的api串联起来，做一个能跑起来的客户端和服务器。
客户端和服务器采用阻塞的同步读写方式完成通信

客户端设计

客户端设计基本思路是根据服务器对端的ip和端口创建一个endpoint，然后创建socket连接这个endpoint，之后就可以用同步读写的方式发送和接收数据了。

```
1.
2. #include <iostream>
3. #include <boost/asio.hpp>
4. using namespace std;
5. using namespace boost::asio::ip;
6. const int MAX_LENGTH = 1024;
7. int main()
8. {
9.     try {
10.         //创建上下文服务
11.         boost::asio::io_context   ioc;
12.         //构造endpoint
13.         tcp::endpoint  remote_ep(address::from_string("127.0.0.1"), 10086);
14.         tcp::socket  sock(ioc);
15.         boost::system::error_code   error = boost::asio::error::host_not_found; ;
16.         sock.connect(remote_ep, error);
17.         if (error) {
18.             cout << "connect failed, code is " << error.value() << " error msg is " << error.message
    ();
19.             return 0;
20.         }
21.
22.         std::cout << "Enter message: ";
23.         char request[MAX_LENGTH];
24.         std::cin.getline(request, MAX_LENGTH);
25.         size_t request_length = strlen(request);
26.         boost::asio::write(sock, boost::asio::buffer(request, request_length));
27.
28.         char reply[MAX_LENGTH];
29.         size_t reply_length = boost::asio::read(sock,
30.             boost::asio::buffer(reply, request_length));
31.         std::cout << "Reply is: ";
32.         std::cout.write(reply, reply_length);
33.         std::cout << "\n";
34.     }
35.     catch (std::exception& e) {
36.         std::cerr << "Exception: " << e.what() << endl;
37.     }
38.     return 0;
39. }
```

服务器

session函数

创建session函数，该函数为服务器处理客户端请求，每当我们获取客户端连接后就调用该函数。在session函数里里进行echo方式的读写，所谓echo就是应答式的处理

```
1. void session(socket_ptr sock) {
2.     try {
3.         for (;;) {
4.             char data[max_length];
5.             memset(data, '\0', max_length);
6.             boost::system::error_code error;
7.             size_t length = sock->read_some(boost::asio::buffer(data, max_length), error);
8.             if (error == boost::asio::error::eof) {
9.                 std::cout << "connection closed by peer" << endl;
10.                break;
11.            }
12.            else if (error) {
13.                throw boost::system::system_error(error);
14.            }
15.            cout << "receive from " << sock->remote_endpoint().address().to_string() << endl;
16.            cout << "receive message is " << data << endl;
17.            //回传信息值
18.            boost::asio::write(*sock, boost::asio::buffer(data, length));
19.        }
20.    }
21.    catch (std::exception& e) {
22.        std::cerr << "Exception in thread: " << e.what() << "\n" << std::endl;
23.    }
24. }
```

server函数

server函数根据服务器ip和端口创建服务器acceptor用来接收数据，用socket接收新的连接，然后为这个socket创建session。

```
1. void server(boost::asio::io_context& io_context, unsigned short port) {
2.     tcp::acceptor a(io_context, tcp::endpoint(tcp::v4(), port));
3.     for (;;) {
4.         socket_ptr socket(new tcp::socket(io_context));
5.         a.accept(*socket);
6.         auto t =  std::make_shared<std::thread>(session, socket);
7.         thread_set.insert(t);
8.     }
9. }
```

创建线程调用session函数可以分配独立的线程用于socket的读写，保证acceptor不会因为socket的读写而阻塞。

同步读写的优劣

1 同步读写的缺陷在于读写是阻塞的，如果客户端对端不发送数据服务器的read操作是阻塞的，这将导致服务器处于阻塞等待状态。
2 可以通过开辟新的线程为新生成的连接处理读写，但是一个进程开辟的线程是有限的，约为2048个线程，在Linux环境可以通过unlimit增加一个进程开辟的线程数，但是线程过多也会导致切换消耗的时间片较多。
3 该服务器和客户端为应答式，实际场景为全双工通信模式，发送和接收要独立分开。
4 该服务器和客户端未考虑粘包处理。
综上所述，是我们这个服务器和客户端存在的问题，为解决上述问题，我们在接下里的文章里做不断完善和改进，主要以异步读写改进上述方案。
当然同步读写的方式也有其优点，比如客户端连接数不多，而且服务器并发性不高的场景，可以使用同步读写的方式。使用同步读写能简化编码难度。

源码连接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3956) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(508) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)